```
import torch
import torch.nn as nn

# Define the dimensions of the input and hidden layer
D = 10
H = 10

# Define a simple autoencoder class
class SimpleAutoencoder(nn.Module):
        def __init__(self, input_dim, hidden_dim):
                super(SimpleAutoencoder, self).__init__()
                self.encoder = nn.Linear(input_dim, hidden_dim)
                self.decoder = nn.Linear(hidden_dim, input_dim)
                self.activation = nn.ReLU()

        def forward(self, x):
                h = self.activation(self.encoder(x))
                x_reconstructed = self.decoder(h)
                return x_reconstructed

# Initialize the network
autoencoder = SimpleAutoencoder(D, H)
```

```
SimpleAutoencoder(
  (encoder): Linear(in_features=10, out_features=10, bias=True)
  (decoder): Linear(in_features=10, out_features=10, bias=True)
  (activation): ReLU()
)
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import numpy as np

class NoisyDataset(Dataset):
        def __init__(self, num_samples=1000, input_dim=10, noise_factor=0.5):
                self.num_samples = num_samples
                self.input_dim = input_dim
                self.noise_factor = noise_factor

                # Generate synthetic data
                self.data = np.random.rand(num_samples, input_dim).astype(np.float32)

                # Normalize the data to be between 0 and 1
                self.data = (self.data - np.min(self.data)) / (np.max(self.data) - np.min(self.data))

        def __len__(self):
                return self.num_samples

        def __getitem__(self, index):
                clean_data = self.data[index]
                noisy_data = clean_data + self.noise_factor * np.random.randn(self.input_dim).astype(np.float32)
                noisy_data = np.clip(noisy_data, 0, 1)    # Clip values to be between 0 and 1
                return clean_data, clean_data


dataset = NoisyDataset(num_samples=1000, input_dim=10, noise_factor=0.5)


# Hyperparameters
num_epochs = 500
batch_size = 64
learning_rate = 1e-3

# Dataloader defined here
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
```

```
# Loss function
criterion = nn.MSELoss()

# Optimizer
optimizer = optim.Adam(autoencoder.parameters(), lr=learning_rate)

loss_epoch = []
# Training loop
for epoch in range(num_epochs):

        for data in dataloader:
                inputs,_ = data    # We don't need the labels for autoencoder training
                # Forward pass
                outputs = autoencoder(inputs)
```

```
                loss = criterion(outputs, inputs)    # Calculate the loss between the output and the original inputs

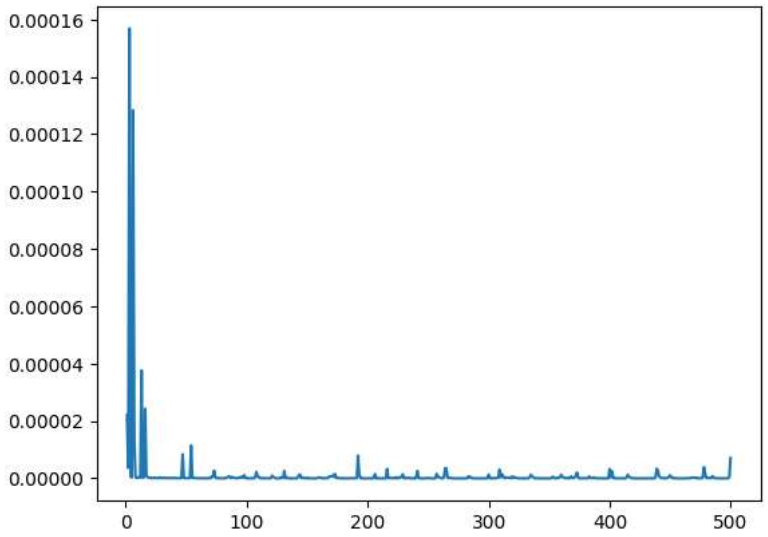
                # Backward and optimize
                optimizer.zero_grad()
                loss.backward()
                optimizer.step()

        loss_epoch.append(loss.item())

loss_epoch = np.array(loss_epoch)
```

```
import matplotlib.pyplot as plt
plt.plot(range(1, len(loss_epoch) + 1), loss_epoch)
```

```
[<matplotlib.lines.Line2D at 0x7cc2b610f2e0>]
```




```
loss_epoch
```

```
array([2.86776483e-01, 2.07917705e-01, 1.42580301e-01, 1.12540700e-01,
       9.34251323e-02, 8.52063447e-02, 7.15045333e-02, 7.66555518e-02,
       7.47427791e-02, 6.65282533e-02, 6.27608523e-02, 6.51037768e-02,
       5.96492402e-02, 6.00489266e-02, 6.25638366e-02, 5.53774126e-02,
       5.20703606e-02, 5.18064462e-02, 4.56837639e-02, 4.24508899e-02,
       4.10954468e-02, 4.31896262e-02, 3.16594876e-02, 3.22170369e-02,
       3.70135829e-02, 3.16844955e-02, 2.69386284e-02, 2.90569551e-02,
       2.76037119e-02, 2.92315315e-02, 2.45926995e-02, 2.63987035e-02,
       2.07602568e-02, 2.17636768e-02, 2.51242667e-02, 2.19227262e-02,
       2.18258146e-02, 2.11281199e-02, 1.96457785e-02, 1.94821265e-02,
       1.72192659e-02, 1.66226681e-02, 1.99205000e-02, 1.69491600e-02,
       1.79326031e-02, 1.37800630e-02, 1.54458620e-02, 1.58474818e-02,
       1.28772296e-02, 9.60879494e-03, 1.04945675e-02, 1.24802962e-02,
       1.11866686e-02, 1.44624747e-02, 1.04988096e-02, 8.75800103e-03,
       9.01759043e-03, 9.95622203e-03, 7.27449777e-03, 9.69430432e-03,
       8.69701058e-03, 8.88047554e-03, 7.80762499e-03, 1.09641468e-02,
       7.10470136e-03, 7.98077695e-03, 7.74636446e-03, 7.35446671e-03,
       8.02850351e-03, 6.97876979e-03, 1.02161476e-02, 8.34182836e-03,
       7.82401860e-03, 6.03429088e-03, 8.08303617e-03, 6.55380078e-03,
       6.91429665e-03, 8.16041697e-03, 6.14794623e-03, 6.75297715e-03,
       5.00594033e-03, 7.47516751e-03, 5.05219260e-03, 5.39846579e-03,
       5.66560216e-03, 5.13133267e-03, 4.71440097e-03, 6.60183793e-03,
       6.33152667e-03, 4.08747001e-03, 4.16569458e-03, 4.10021702e-03,
       3.50055238e-03, 3.17945355e-03, 2.74306000e-03, 1.64315326e-03,
       2.74024135e-03, 2.31042528e-03, 2.37221224e-03, 1.77957234e-03,
       1.14653772e-03, 1.62392622e-03, 1.16167811e-03, 8.22812086e-04,
       9.72139591e-04, 6.36040291e-04, 6.52780174e-04, 3.91058740e-04,
       4.09510569e-04, 5.18103596e-04, 4.82374104e-04, 2.22399918e-04,
       2.09506616e-04, 1.49233412e-04, 1.81422787e-04, 1.46851584e-04,
       1.03174934e-04, 7.79778638e-05, 1.49457614e-04, 9.15085184e-05,
       5.87401337e-05, 4.83364602e-05, 1.12386922e-04, 3.36025078e-05,
       5.80580236e-05, 4.93122934e-05, 2.87793646e-05, 1.59581850e-05,
       2.11546958e-05, 7.94974039e-05, 3.27246817e-05, 3.32923300e-05,
       1.67143589e-04, 1.19848792e-05, 1.61031676e-05, 1.10011060e-05,
       2.19571630e-05, 3.75044474e-05, 7.71105188e-05, 1.96134788e-05,
       8.39746499e-05, 1.06752923e-05, 6.79444520e-06, 6.98006697e-05,
       4.37755807e-06, 6.71819435e-05, 4.95944187e-06, 1.32983687e-05,
       1.47892079e-05, 4.01131820e-06, 3.55105681e-06, 3.12229736e-06,
       3.51387371e-06, 6.61374215e-05, 2.16186818e-05, 2.64920004e-06,
       2.22427207e-05, 2.30669229e-06, 1.60597738e-05, 1.43286898e-05,
       2.24976102e-05, 4.82434416e-06, 1.25479746e-05, 1.33272306e-06,
       1.30932835e-06, 6.11022624e-05, 1.01097030e-05, 1.83341890e-05,
       1.61537207e-06, 1.86798134e-05, 1.73587603e-06, 1.61473156e-06,
       8.88614181e-07, 8.43058660e-06, 3.07669302e-06, 3.01814980e-05,
       2.91672459e-05, 2.42315691e-05, 5.94270750e-05, 1.43160287e-06,
       1.07576000e-06, 4.37015103e-07, 4.51728113e-07, 4.11408109e-07,
       1.04122319e-06, 8.99529482e-07, 2.56942343e-07, 2.10222424e-05,
```

```
3.54018312e-07, 1.34251222e-05, 1.65966012e-05, 1.39877593e-05,
1.56121441e-05, 5.18648358e-06, 2.30065348e-06, 6.91284367e-06,
1.64218747e-07, 1.03550847e-05, 6.64972276e-06, 1.27416642e-05,
1.79737126e-05, 1.13839697e-07, 9.50923095e-06, 1.61707817e-07,
1.08001746e-06, 8.45113561e-08, 1.30375980e-07, 6.22678371e-08,
7.06777428e-08, 9.80039729e-08, 1.12882083e-06, 1.48880383e-06,
2.46119384e-06, 5.75302010e-06, 3.41942223e-06, 5.88971218e-08,
6.31906190e-08, 5.57336790e-08, 6.28780512e-08, 2.55429109e-06,
1.11561421e-05, 4.23437143e-08, 3.78611526e-06, 5.50478205e-08,
4.15056002e-06, 3.75802358e-08, 3.35876575e-06, 8.46902276e-06,
3.34581091e-06, 3.87173849e-08, 3.40659341e-08, 2.71512477e-06,
```

1 开始借助 AI 编写或生成代码。